

Task 20: Data Types and Type Conversion

Track: Data Science | Level 1 - Day 20 | Tools: Python, Pandas, Jupyter Notebook

Level 1 - Day 20: Data Types and Type Conversion in Pandas

Track: Data Science

Task: Task 20 - Data Types and Type Conversion

Objective: Inspect DataFrame data types, understand the impact of correct data typing on performance and analytics, and convert columns between strings, integers, floats, booleans, and datetime objects using ``astype()``, ``pd.to_numeric()``, and ``pd.to_datetime()``.

1. Quick Reference: Pandas Data Types & Conversion Functions

Target Data Type	Primary Method	Common Use Cases & Benefits
Integer (`int64` / `Int64`)	<code>`df['col'].astype('int64')`</code> or <code>`pd.to_numeric(..., downcast='integer')`</code>	Counts, discrete quantities, IDs; enables nullable <code>`Int64`</code>
Float (`float64`)	<code>`pd.to_numeric(df['col'], errors='coerce')`</code>	Financial metrics, continuous measurements; handles dirty strings
Boolean (`bool`)	<code>`df['col'].astype(bool)`</code> or <code>`.map({'Yes': True, 'No': False})`</code>	Binary flags, active status, logic masking
Datetime (`datetime64[ns]`)	<code>`pd.to_datetime(df['col'], format='%Y-%m-%d')`</code>	Time-series analysis, feature extraction (year, month, day)
Category (`category`)	<code>`df['col'].astype('category')`</code>	Low-cardinality strings; reduces memory footprint by up to 90%

2. Setting Up a Raw Dataset with Inconsistent & Mismatched Data Types

In [code]

```
import pandas as pd
import numpy as np

# Simulating uncleaned raw data as ingested from dirty CSV/JSON files
raw_data = {
    'StudentID': ['1001', '1002', '1003', '1004', '1005', '1006'],
    'Student_Name': ['Aman Verma', 'Pooja Patel', 'Rohan Joshi', 'Sneha Sen', 'Vikas Rao', 'Aditi
Roy'],
    'Age': ['21', '22', '20', '23', '21', '22'], # Stored as string
    'Fee_Paid': ['1', '0', '1', '1', '0', '1'], # Numeric string instead of
boolean
    'Tuition_Fee': ['$4,500.50', '$5,200.00', '$4,800.75', '$5,100.00', 'Pending', '$4,950.25'], #
Dirty currency strings
    'Exam_Score': ['88.5', '92.0', '74.5', 'invalid', '65.0', '98.0'], # Mixed float strings with
invalid entries
    'Enrollment_Date': ['2026-01-15', '2026-02-01', '2026-01-20', '2026-03-05', '2026-01-10',
'2026-02-18'], # Date stored as object
    'Department': ['Data Science', 'Data Science', 'Tech', 'Tech', 'Data Science', 'Tech'] # Repeated
categorical strings
}

df = pd.DataFrame(raw_data)
print("=== RAW DATASET SNAPSHOT ===")
df
```

3. Initial Data-Type Inspection Report

Inspecting column data types and checking memory usage using `.dtypes` and .info().`

In [code]

```
# Generate formal Data-Type Inspection Report
inspection_report = pd.DataFrame({
    'Column_Name': df.columns,
    'Current_Dtype': df.dtypes.values,
    'Sample_Value': [df[col].iloc[0] for col in df.columns],
    'Expected_Target_Type': ['Integer', 'String', 'Integer', 'Boolean', 'Float', 'Float', 'Datetime',
'Category']
})

print("=== INITIAL DATA-TYPE INSPECTION REPORT ===")
inspection_report
```

In [code]

```
# Check memory usage before conversions
initial_memory = df.memory_usage(deep=True).sum()
print(f"Initial Total DataFrame Memory Footprint: {initial_memory:,} bytes")
```

4. Converting at Least Four Column Types (Demonstrations)

1. **Conversion 1 (String to Integer):** Converting ``Age`` and ``StudentID`` using ``astype(int)``.

2. **Conversion 2 (String to Boolean):** Converting ``Fee_Paid`` using ``astype(int).astype(bool)``.

3. **Conversion 3 (Dirty String to Float):** Sanitizing currency strings and coercing errors with `pd.to_numeric(..., errors='coerce')`.
4. **Conversion 4 (String to Datetime):** Converting `'Enrollment_Date'` with `pd.to_datetime()` and extracting temporal features.
5. **Bonus Conversion (String to Category):** Optimizing `'Department'` with `.astype('category')`.

In [code]

```
df_clean = df.copy()

# 1. Convert Age & StudentID to Integer
df_clean['StudentID'] = df_clean['StudentID'].astype('int64')
df_clean['Age'] = df_clean['Age'].astype('int32')

# 2. Convert Fee_Paid to Boolean
df_clean['Fee_Paid'] = df_clean['Fee_Paid'].astype(int).astype(bool)

# 3. Convert Dirty Currency & Exam_Score to Float using pd.to_numeric with error coercion
# Clean '$' and commas from Tuition_Fee first
df_clean['Tuition_Fee'] = df_clean['Tuition_Fee'].str.replace('$', '', regex=False).str.replace(',', '', regex=False)
df_clean['Tuition_Fee'] = pd.to_numeric(df_clean['Tuition_Fee'], errors='coerce')

# Coerce 'invalid' in Exam_Score to NaN and fill with column mean
df_clean['Exam_Score'] = pd.to_numeric(df_clean['Exam_Score'], errors='coerce')
df_clean['Exam_Score'] = df_clean['Exam_Score'].fillna(round(df_clean['Exam_Score'].mean(), 1))

# 4. Convert Enrollment_Date to Datetime
df_clean['Enrollment_Date'] = pd.to_datetime(df_clean['Enrollment_Date'], format='%Y-%m-%d')

# 5. Convert Department to Categorical
df_clean['Department'] = df_clean['Department'].astype('category')

print("=== CONVERSIONS COMPLETE ===")
```

5. Cleaned Dataset Verification & Comparison

In [code]

```
# Post-conversion data types inspection
post_inspection = pd.DataFrame({
    'Column_Name': df_clean.columns,
    'Original_Dtype': df.dtypes.values,
    'Converted_Dtype': df_clean.dtypes.values,
    'Sample_Converted_Value': [df_clean[c].iloc[0] for c in df_clean.columns]
})

print("=== DATA TYPE COMPARISON REPORT ===")
post_inspection
```

In [code]

```
# Final Cleaned Dataset Snapshot
print("=== CLEANED DATASET SNAPSHOT ===")
df_clean
```

In [code]

```
# Memory savings comparison
cleaned_memory = df_clean.memory_usage(deep=True).sum()
savings_pct = ((initial_memory - cleaned_memory) / initial_memory) * 100

print(f"Initial Memory Usage: {initial_memory:,} bytes")
print(f"Cleaned Memory Usage: {cleaned_memory:,} bytes")
print(f"Total Memory Savings: {savings_pct:.2f}%")
```

6. Downstream Analytics Enabled by Proper Typing

Demonstrating date math, numerical aggregation, and boolean masking operations that fail or produce errors on raw string data.

In [code]

```
# 1. Datetime feature extraction
df_clean['Enrollment_Month'] = df_clean['Enrollment_Date'].dt.month_name()
df_clean['Enrollment_Quarter'] = df_clean['Enrollment_Date'].dt.quarter

# 2. Mathematical aggregations
avg_score = df_clean['Exam_Score'].mean()
total_tuition = df_clean['Tuition_Fee'].sum()

# 3. Boolean filtering
unpaid_students = df_clean[~df_clean['Fee_Paid']]

print(f"Average Cohort Exam Score: {avg_score:.2f}%")
print(f"Total Collected Tuition: ${total_tuition:,.2f}")
print(f"Students with Pending Fee: {list(unpaid_students['Student_Name'])}")

print("\n=== FINAL ENRICHED DATASET ===")
df_clean[['StudentID', 'Student_Name', 'Age', 'Department', 'Exam_Score', 'Fee_Paid',
'Enrollment_Month']]
```